
FPGA Experiment Report

Experiment 5 — PS/2 Keyboard Interface

Task: FPGA 5 — Keyboard (PS/2 interface)

Student 1: Mahmood Stitia

ID: 214032682

Student 2: Saleh Khalil

ID: 213310485

Part 1 — Status update

The table below lists the submitted files and their status. The status is one of *Done* (module and simulation written and passing), *Partially done* (started but not fully working — explained in the notes), or *Not started*.

File / Step	Status	Notes / comments
Ps2_Interface.v	Done	
Ps2_Interface_tb.v	Done	
Ps2_Display.v	Done	
Ps2_Top.v	Done	
task1.xdc	Done	

Issues

Any errors present at the time of submission are described here (per module / simulation, with what the error means).

- Ps2_Interface.v:
- Ps2_Interface_tb.v:
- Ps2_Display.v:
- Ps2_Top.v:
- task1.xdc:

Part 2 — Simulations and answers

Simulation — Ps2_Interface

To verify the `Ps2_Interface` module, we emulate the behavior of a real PS/2 keyboard. The PS/2 clock is generated with a 60 period (a 30 half-period), which corresponds to a frequency of approximately 16.7. This value sits inside the standard PS/2 clock range (1016.7), so the simulation exercises the interface at a realistic device speed.

Each key event is transmitted by the `send_byte` task, which serializes a byte into the standard 11-bit PS/2 frame: one start bit (0), eight data bits sent LSB-first, one odd-parity bit, and one stop bit (1). For every bit the data line is updated and the PS/2 clock is toggled low-then-high, reproducing the way a keyboard shifts data out on its own clock. A short idle gap is inserted between frames to separate consecutive presses.

Simulated Scenario

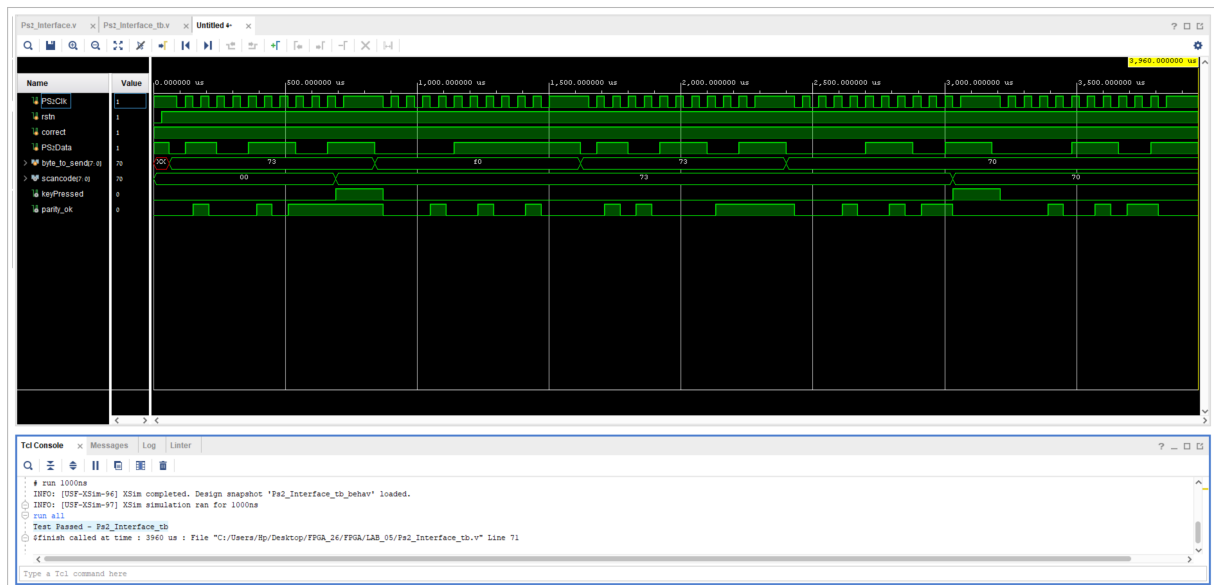
The testbench plays out three scenarios that cover the key behaviours of the interface:

1. **Press '5' (0x73).** A single make-code is sent. We expect `keyPressed` to pulse and `scancode` to equal 0x73.
2. **Release '5' then press '0'.** First the break sequence F0 73 is sent (the F0 prefix marks a key release), and the interface correctly ignores it without raising `keyPressed`. Immediately afterwards the make-code 0x70 ('0') is sent, and we expect `keyPressed` to pulse with `scancode == 0x70`. This checks that a release of one key is discarded and a following new press is still detected.
3. **Hold '0' (auto-repeat).** The same make-code 0x70 is sent a second time with no intervening release. Because the key is already considered pressed, the interface must *not* raise `keyPressed` again, while `scancode` still reads 0x70. This verifies that the keyboard's typematic auto-repeat does not generate spurious new presses.

Metastability and Clock-Domain Considerations

The PS/2 clock and data lines come from the keyboard and are *asynchronous* to the FPGA. Sampling such a signal while it is transitioning can drive a flip-flop into a *metastable* state, where its output settles to an undefined value and may corrupt downstream logic.

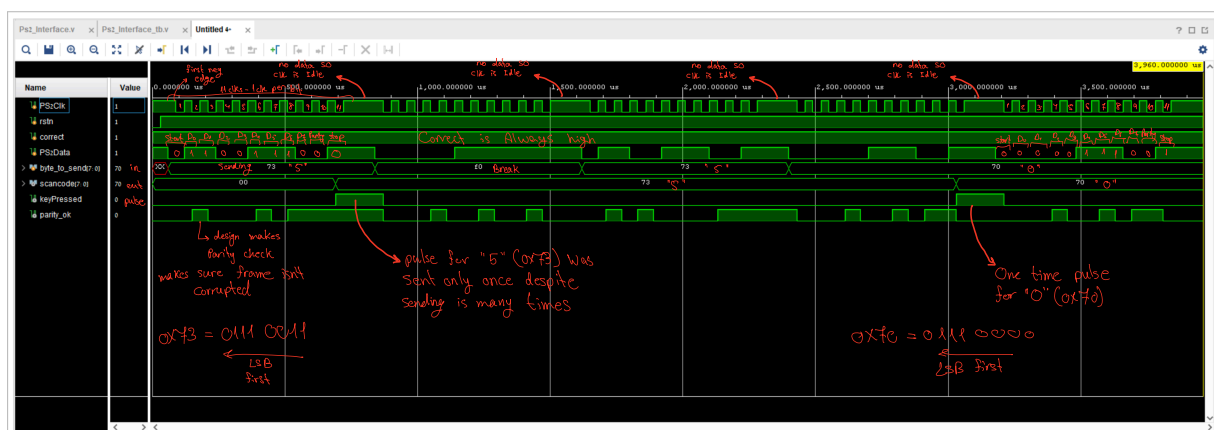
Following the approach from class, we only sample the line when it is guaranteed stable. The PS/2 protocol holds `PS2Data` steady through the falling edge of `PS2Clk`, so our receiver shifts the data in on `negedge PS2Clk`, capturing each bit in its stable window and avoiding metastable sampling.



Full Ps2_Interface_tb with TCl Console message **Test Passed - Ps2_Interface_tb**

Signals (top to bottom in the waveform):

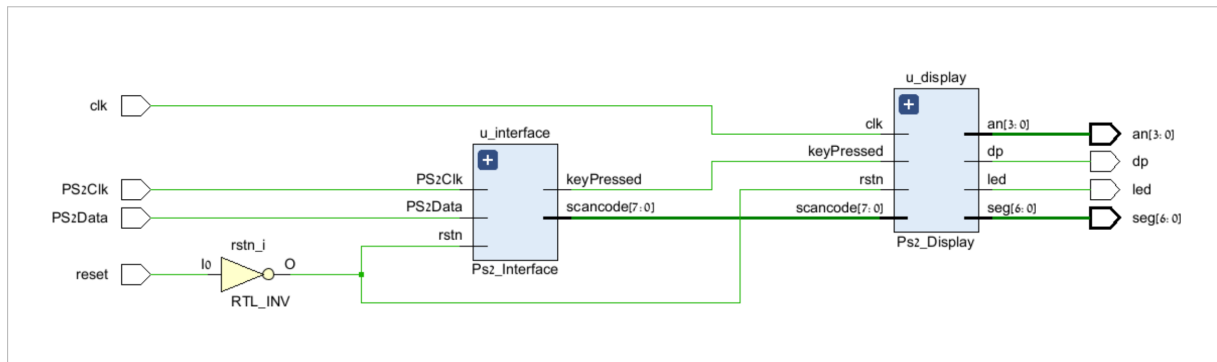
- PS2C1k - the keyboard clock (input). It toggles only while a byte is being sent and is idle-high.
- rstn - active-low *asynchronous* reset (driven by the test-bench).
- correct - a test-bench-only pass/fail flag.
- PS2Data - the serial data line. Carries 11-bit frame, sampled on the falling edge of PS2C1k.
- byte_to_send[7:0] - a test-bench register holding the byte currently being bit-banged onto PS2Data.
- scancode[7:0] - interface output: the most recent accepted make-code.
- keyPressed - interface output: a one-cycle pulse on the first make-code of a new press.
- parity_ok - internal interface signal : high when the frame's odd-parity bit matches the 8 data bits.



Annotated close-up of the same simulation. The red notes mark the first key edge and that the first bit is sent on the negative clock edge.

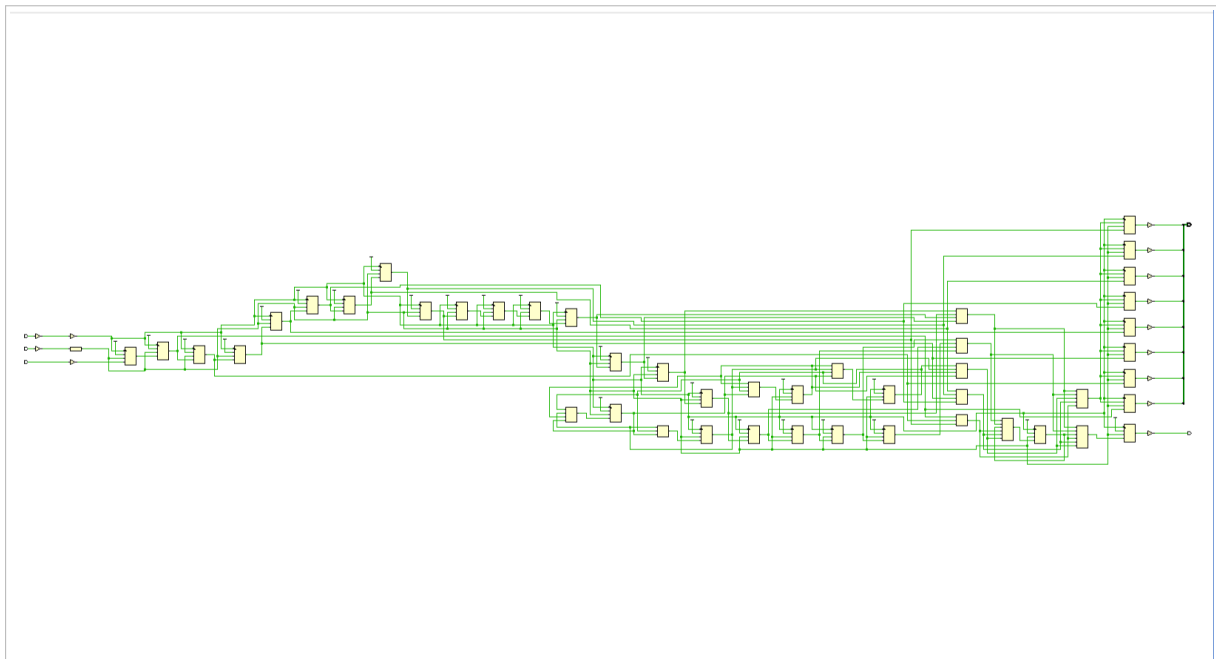
Elaborated Design

Top module



RTL-elaborated schematic of `Ps2_Top`: `u_interface` (`Ps2_Interface`, `PS2Cik` domain) feeds `keyPressed` and `scancode[7:0]` into `u_display` (`Ps2_Display`, 100 MHz domain); the reset push-button passes through an `RTL_INV` (`rstn_i`) into `rstn`. Inputs `clk`, `PS2Cik`, `PS2Data`, `reset` are on the left and outputs `an[3:0]`, `dp`, `led`, `seg[6:0]` on the right.

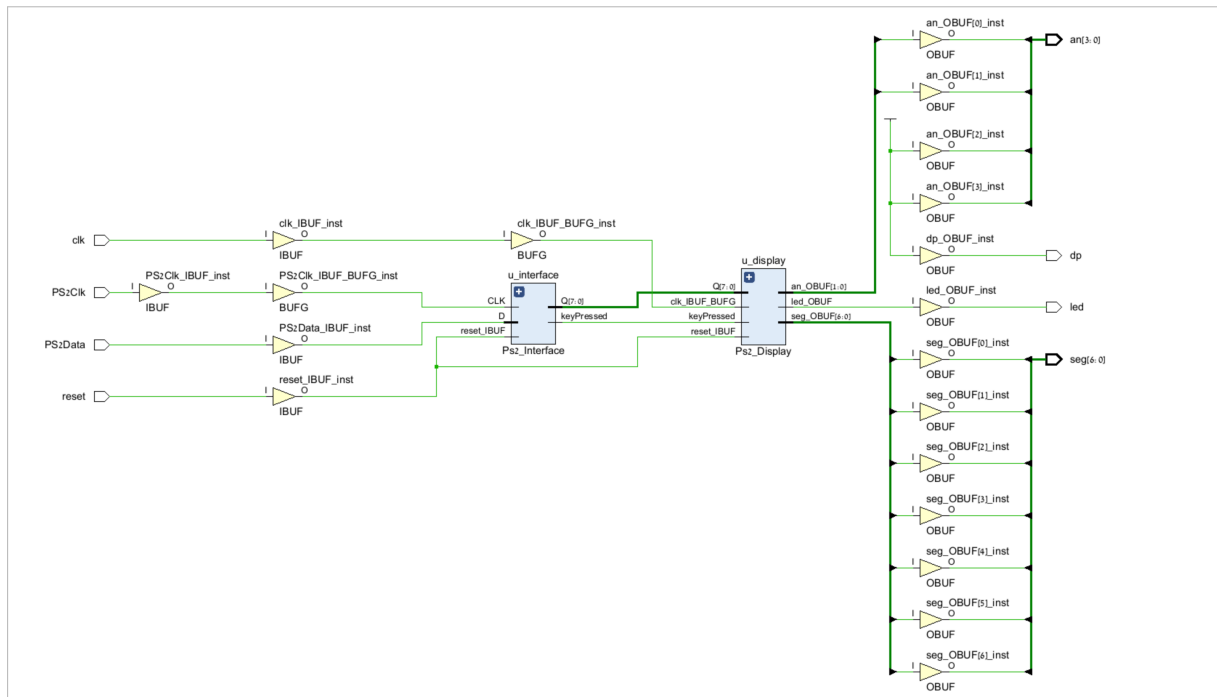
Ps2_Interface



RTL-elaborated schematic of the Interface

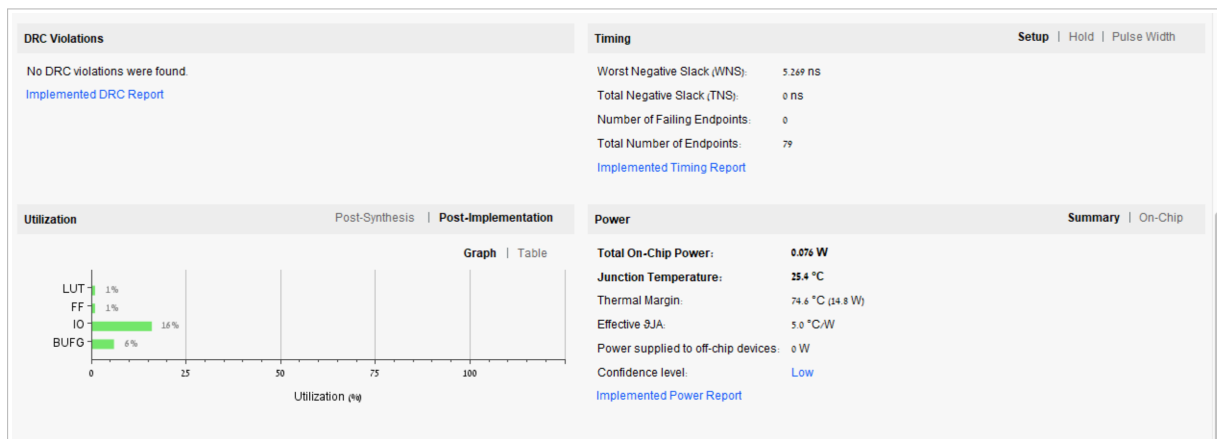
Implementation

Synthesis schematic



Post-synthesis schematic. Every external port now passes through an IBUF/OBUF, the two clocks

Project summary



Post-implementation project summary

Timing summary

Design Timing Summary					
Setup		Hold	Pulse Width		
Worst Negative Slack (WNS):	5.269 ns	Worst Hold Slack (WHS):	0.330 ns	Worst Pulse Width Slack (WPWS):	4.500 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	79	Total Number of Endpoints:	79	Total Number of Endpoints:	57
All user specified timing constraints are met.					

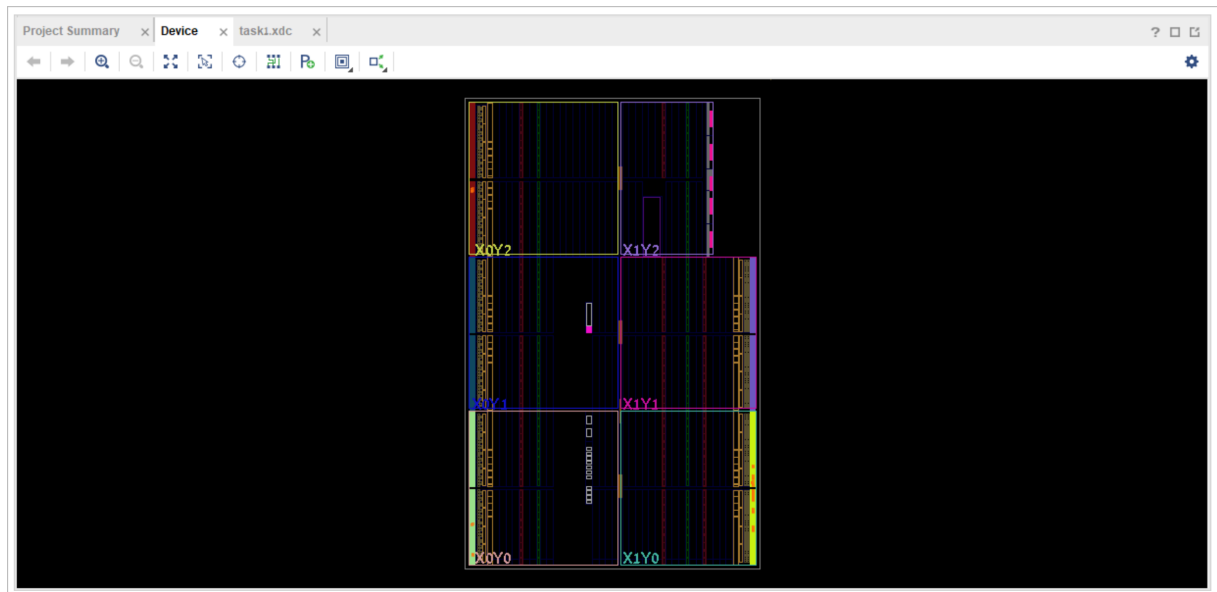
Design Timing Summary after implementation

Critical Path

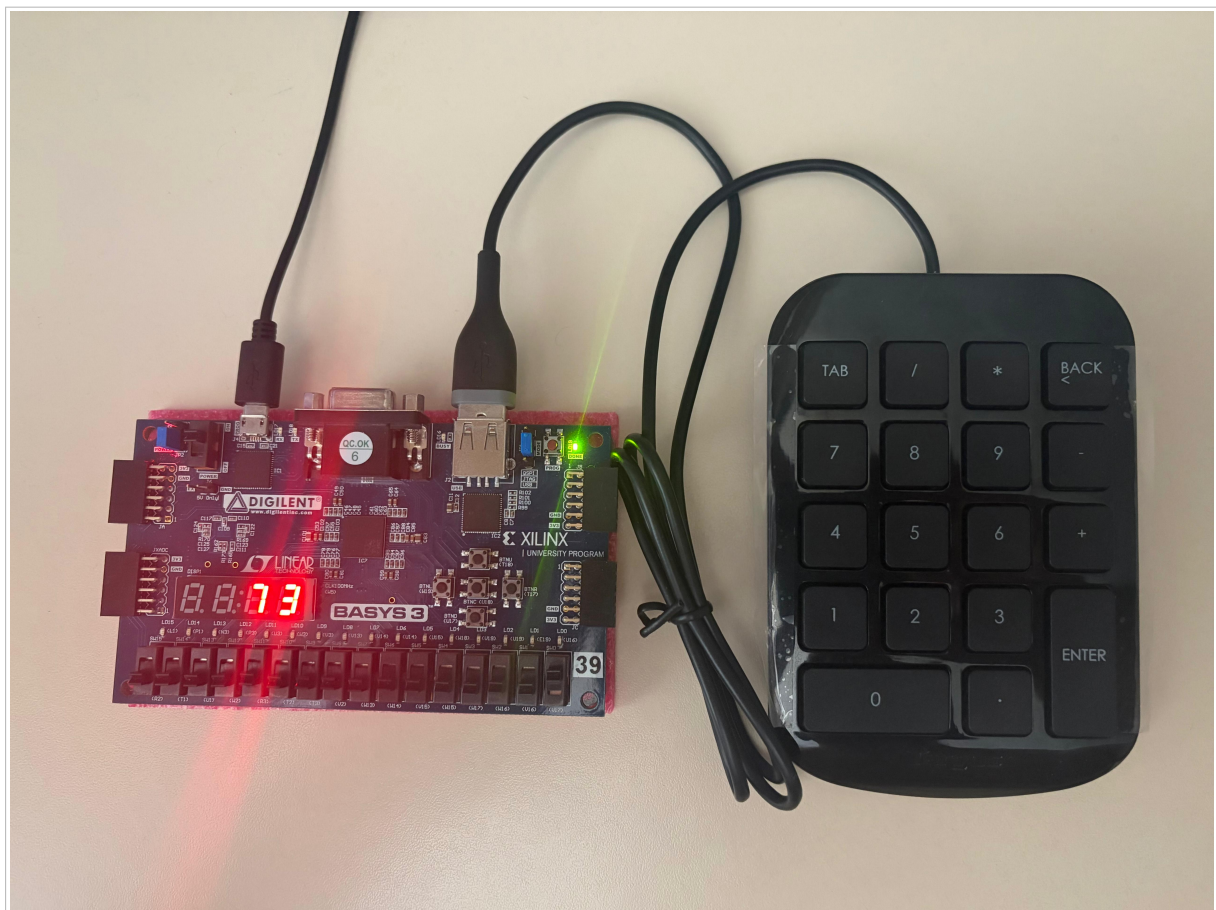
Name	Slack	Levels	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception	Clock t
Path 1	5.269	3	25	u_displayled_timer_reg[16]/C	u_displayled_timer_reg[20]/CE	4.361	1.054	3.307	10.0	sys_clk_pin	sys_clk_pin		
Path 2	5.269	3	25	u_displayled_timer_reg[16]/C	u_displayled_timer_reg[21]/CE	4.361	1.054	3.307	10.0	sys_clk_pin	sys_clk_pin		
Path 3	5.269	3	25	u_displayled_timer_reg[16]/C	u_displayled_timer_reg[22]/CE	4.361	1.054	3.307	10.0	sys_clk_pin	sys_clk_pin		
Path 4	5.269	3	25	u_displayled_timer_reg[16]/C	u_displayled_timer_reg[23]/CE	4.361	1.054	3.307	10.0	sys_clk_pin	sys_clk_pin		
Path 5	5.534	3	25	u_displayled_timer_reg[16]/C	u_displayled_timer_reg[4]/CE	4.100	1.054	3.046	10.0	sys_clk_pin	sys_clk_pin		
Path 6	5.534	3	25	u_displayled_timer_reg[16]/C	u_displayled_timer_reg[5]/CE	4.100	1.054	3.046	10.0	sys_clk_pin	sys_clk_pin		
Path 7	5.534	3	25	u_displayled_timer_reg[16]/C	u_displayled_timer_reg[6]/CE	4.100	1.054	3.046	10.0	sys_clk_pin	sys_clk_pin		
Path 8	5.534	3	25	u_displayled_timer_reg[16]/C	u_displayled_timer_reg[7]/CE	4.100	1.054	3.046	10.0	sys_clk_pin	sys_clk_pin		
Path 9	5.576	3	25	u_displayled_timer_reg[16]/C	u_displayled_timer_reg[16]/CE	4.087	1.054	3.033	10.0	sys_clk_pin	sys_clk_pin		
Path 10	5.576	3	25	u_displayled_timer_reg[16]/C	u_displayled_timer_reg[17]/CE	4.087	1.054	3.033	10.0	sys_clk_pin	sys_clk_pin		

Intra-Clock Paths report. The worst paths (Path 1–4) run inside u_display from led_timer_reg[16]/C to led_timer_reg[20..23]/CE, with slack 5.269 ns, 3 logic levels and 4.361 ns total delay (1.054 ns logic + 3.307 ns net) against the 10 ns requirement

Device



Device (floorplan) view of the placed-and-routed design on the Artix-7 (XC7A35T)



The full system on hardware: the numeric keypad plugged into the Basys-3 USB-host port. Key "5" is pressed and its PS/2 scan-code 73 is shown on the two right-hand 7-segment digits.

Appendix — Verilog Source Code

The full source of the submitted files is listed below. Paste the code of each file inside its block.

Ps2_Interface.v

```

1  module Ps2_Interface(
2      input wire PS2Clk,
3      input wire rstn,
4      input wire PS2Data,
5      output reg [7:0] scancode,
6      output reg keyPressed
7  );
8
9      reg [21:0] shift_reg;
10     reg [3:0] bit_count;
11     reg is_valid;
12     wire parity_ok;
13     wire [7:0] cur_byte = shift_reg[20:13];
14     wire [7:0] prev_byte = shift_reg[9:2];
15
16     always @(negedge PS2Clk or negedge rstn) begin
17         if (!rstn) begin
18             bit_count <= 4'b0;
19             shift_reg <= 22'b0;
20             scancode <= 8'b0;
21             keyPressed <= 1'b0;
22             is_valid <= 1'b1;
23         end else begin
24             shift_reg <= {PS2Data, shift_reg[21:1]};
25             bit_count <= (bit_count == 4'd10)? 4'd0 : bit_count + 4'd1;
26             keyPressed <= 1'b0; // default
27
28             if (bit_count == 4'd10) begin
29                 if (cur_byte == 8'hE0 || cur_byte == 8'hF0) begin
30                     // skip
31                 end
32                 else if (prev_byte == 8'hF0) begin
33                     is_valid <= 1'b1;
34                 end
35                 else begin
36                     scancode <= cur_byte;
37                     if (is_valid) begin
38                         keyPressed <= parity_ok;
39                         is_valid <= 1'b0;
40                     end
41                 end
42             end
43
44         end
45
46     end
47
48     assign parity_ok = (shift_reg[21] == ~(^cur_byte));
49
50 endmodule

```


Ps2_Interface_tb.v

```

1  module Ps2_Interface_tb();
2
3      localparam CLK_HALF = 30_000;    // 30 us half -> 60 us period -> ~16.7
        kHz PS/2 clock
4
5      reg PS2Clk, rstn, PS2Data, correct;
6      wire [7:0] scancode;
7      wire keyPressed;
8      wire parity_ok = uut.parity_ok;
9      reg [7:0] byte_to_send;
10
11
12      // Instantiate the UUT (Unit Under Test)
13      Ps2_Interface uut(
14          .PS2Clk(PS2Clk),
15          .rstn(rstn),
16          .PS2Data(PS2Data),
17          .scancode(scancode),
18          .keyPressed(keyPressed)
19      );
20
21      initial begin
22
23          if($test$plusargs("vcd")) begin
24              $dumpfile("Ps2_Interface_tb.vcd");
25              $dumpvars(0, Ps2_Interface_tb);
26          end
27
28          correct = 1;
29          PS2Clk = 1;
30          PS2Data = 1;
31          rstn = 0;
32
33          #(CLK_HALF);
34          rstn = 1;
35          #(CLK_HALF);
36
37          // 1- press '5' (0x73)
38          byte_to_send = 8'h73;
39          send_byte(byte_to_send);
40          correct = correct & keyPressed & (scancode == 8'h73);
41
42          // 2- release '5' (F0 73) then press '0' (0x70)
43          byte_to_send = 8'hF0;
44          send_byte(byte_to_send);
45          byte_to_send = 8'h73;
46          send_byte(byte_to_send);
47          byte_to_send = 8'h70;
48          send_byte(byte_to_send);
49          correct = correct & keyPressed & (scancode == byte_to_send);
50
51          // 3- hold '0' (auto-repeat 0x70)
52          send_byte(byte_to_send);
53          correct = correct & ~keyPressed & (scancode == byte_to_send);
54
55
56          if (correct)
57              $display("Test Passed - %m");
58          else
59              $display("Test Failed - %m");
60          $finish;

```

```
61     end
62
63     // send the pressed key in serial format - 11 bits
64     task send_byte(input [7:0] b);
65         integer k;
66         reg [10:0] frame;
67         begin
68             frame = {1'b1, ~(~b), b, 1'b0};
69             for (k = 0; k < 11; k = k + 1) begin
70                 PS2Data = frame[k];
71                 #CLK_HALF;
72                 PS2Clk = 0;
73                 #CLK_HALF;
74                 PS2Clk = 1;
75             end
76             #(CLK_HALF*4); // idle time gap between presses
77         end
78     endtask
79
80 endmodule
```

Ps2_Display.v

```

1  module Ps2_Display(
2      input wire      clk,          // 100 MHz system clock
3      input wire      rstn,         // active-low reset
4      input wire      keyPressed,   // pulse from Ps2_Interface (slow domain)
5      input wire [7:0] scancode,    // byte from Ps2_Interface (slow domain)
6      output wire [6:0] seg,        // 7-segment cathodes (active low)
7      output wire [3:0] an,         // 4 digit anodes (active low)
8      output wire      dp,          // decimal point
9      output reg       led          // strobe LED, one visible blink per
        press
10 );
11 reg [6:0] a_to_g;
12 reg [3:0] an_reg;
13
14 assign seg = a_to_g;
15 assign an  = an_reg;
16 assign dp  = 1'b1;
17
18 wire [1:0] s;
19 reg  [3:0] digit;
20
21 reg kp_sync1, kp_sync2, kp_prev;
22 reg [7:0] valid_scancode;
23
24 always @(posedge clk or negedge rstn) begin
25     if (!rstn) begin
26         kp_sync1 <= 1'b0;
27         kp_sync2 <= 1'b0;
28         kp_prev  <= 1'b0;
29         valid_scancode <= 8'h00;
30     end else begin
31         kp_sync1 <= keyPressed;
32         kp_sync2 <= kp_sync1;
33         kp_prev  <= kp_sync2;
34
35         if (kp_sync2 & ~kp_prev) begin
36             valid_scancode <= scancode;
37         end
38     end
39 end
40
41 wire kp_edge = kp_sync2 & ~kp_prev;
42
43 reg [23:0] led_timer;
44
45 always @(posedge clk or negedge rstn) begin
46     if (!rstn) begin
47         led_timer <= 24'd0;
48         led <= 1'b0;
49     end else if (kp_edge) begin
50         led_timer <= 24'd10_000_000; // ~0.1 seconds at 100MHz
51         led <= 1'b1;
52     end else if (led_timer > 0) begin
53         led_timer <= led_timer - 1;
54         led <= 1'b1;
55     end else begin
56         led <= 1'b0;
57     end
58 end
59
60 reg [19:0] clkdiv = 20'b0;

```

```

61
62     always @(posedge clk or negedge rstn) begin
63         if (!rstn)
64             clkdiv <= 0;
65         else
66             clkdiv <= clkdiv + 1;
67     end
68
69     assign s = clkdiv[19:18];
70
71     always @(*) begin
72         an_reg = 4'b1111;
73
74         case(s)
75             2'b00: begin
76                 digit = valid_scancode[3:0];
77                 an_reg[0] = 1'b0;
78             end
79             2'b01: begin
80                 digit = valid_scancode[7:4];
81                 an_reg[1] = 1'b0;
82             end
83             default: begin
84                 digit = 4'h0;
85                 an_reg = 4'b1111;
86             end
87         endcase
88     end
89
90     always @(*) begin
91         case(digit)
92             //////////////////////////////////<---MSB-LSB<---////////////////////////////////
93             4'h0: a_to_g = 7'b1000000; // 0
94             4'h1: a_to_g = 7'b1111001; // 1
95             4'h2: a_to_g = 7'b0100100; // 2
96             4'h3: a_to_g = 7'b0110000; // 3
97             4'h4: a_to_g = 7'b0011001; // 4
98             4'h5: a_to_g = 7'b0010010; // 5
99             4'h6: a_to_g = 7'b0000010; // 6
100            4'h7: a_to_g = 7'b1111000; // 7
101            4'h8: a_to_g = 7'b0000000; // 8
102            4'h9: a_to_g = 7'b0010000; // 9
103            4'hA: a_to_g = 7'b0001000; // A
104            4'hB: a_to_g = 7'b0000011; // b
105            4'hC: a_to_g = 7'b1000110; // C
106            4'hD: a_to_g = 7'b0100001; // d
107            4'hE: a_to_g = 7'b0000110; // E
108            4'hF: a_to_g = 7'b0001110; // F
109            default: a_to_g = 7'b1111111; // blank
110        endcase
111    end
112 endmodule

```

Ps2_Top.v

```
1 module Ps2_Top(  
2     input wire      clk,           // W5 - 100 MHz system clock  
3     input wire      reset,        // U18 - btnC, active high (pressed = 1)  
4     input wire      PS2Clk,       // C17 - keyboard clock  
5     input wire      PS2Data,      // B17 - keyboard data  
6     output wire [6:0] seg,        // 7-segment cathodes  
7     output wire [3:0] an,         // 7-segment anodes  
8     output wire      dp,          // decimal point  
9     output wire      led          // U16 - strobe LED (LD0)  
10 );  
11  
12 wire rstn = ~reset;  
13 wire keyPressed;  
14 wire [7:0] scancode;  
15  
16 Ps2_Display u_display(  
17     .clk (clk),  
18     .rstn (rstn),  
19     .keyPressed (keyPressed),  
20     .scancode (scancode),  
21     .seg (seg),  
22     .an (an),  
23     .dp (dp),  
24     .led (led));  
25  
26 Ps2_Interface u_interface(  
27     .PS2Clk (PS2Clk),  
28     .rstn (rstn),  
29     .PS2Data (PS2Data),  
30     .scancode (scancode),  
31     .keyPressed (keyPressed));  
32  
33 endmodule
```

task1.xdc

```

1  # Clock signal
2  set_property PACKAGE_PIN W5 [get_ports clk]
3  set_property IOSTANDARD LVCMOS33 [get_ports clk]
4  create_clock -period 10.000 -name sys_clk_pin -waveform {0.000 5.000} -add [
    get_ports clk]
5  # LEDs
6  set_property PACKAGE_PIN U16 [get_ports led]
7  set_property IOSTANDARD LVCMOS33 [get_ports led]
8  #7 segment display
9  set_property PACKAGE_PIN W7 [get_ports {seg[0]}]
10 set_property IOSTANDARD LVCMOS33 [get_ports {seg[0]}]
11 set_property PACKAGE_PIN W6 [get_ports {seg[1]}]
12 set_property IOSTANDARD LVCMOS33 [get_ports {seg[1]}]
13 set_property PACKAGE_PIN U8 [get_ports {seg[2]}]
14 set_property IOSTANDARD LVCMOS33 [get_ports {seg[2]}]
15 set_property PACKAGE_PIN V8 [get_ports {seg[3]}]
16 set_property IOSTANDARD LVCMOS33 [get_ports {seg[3]}]
17 set_property PACKAGE_PIN U5 [get_ports {seg[4]}]
18 set_property IOSTANDARD LVCMOS33 [get_ports {seg[4]}]
19 set_property PACKAGE_PIN V5 [get_ports {seg[5]}]
20 set_property IOSTANDARD LVCMOS33 [get_ports {seg[5]}]
21 set_property PACKAGE_PIN U7 [get_ports {seg[6]}]
22 set_property IOSTANDARD LVCMOS33 [get_ports {seg[6]}]
23
24 set_property PACKAGE_PIN V7 [get_ports dp]
25 set_property IOSTANDARD LVCMOS33 [get_ports dp]
26
27 set_property PACKAGE_PIN U2 [get_ports {an[0]}]
28 set_property IOSTANDARD LVCMOS33 [get_ports {an[0]}]
29 set_property PACKAGE_PIN U4 [get_ports {an[1]}]
30 set_property IOSTANDARD LVCMOS33 [get_ports {an[1]}]
31 set_property PACKAGE_PIN V4 [get_ports {an[2]}]
32 set_property IOSTANDARD LVCMOS33 [get_ports {an[2]}]
33 set_property PACKAGE_PIN W4 [get_ports {an[3]}]
34 set_property IOSTANDARD LVCMOS33 [get_ports {an[3]}]
35
36
37 #Buttons
38 set_property PACKAGE_PIN U18 [get_ports reset]
39 set_property IOSTANDARD LVCMOS33 [get_ports reset]
40 #USB HID (PS/2)
41 set_property PACKAGE_PIN C17 [get_ports PS2Clk]
42 set_property IOSTANDARD LVCMOS33 [get_ports PS2Clk]
43 set_property PULLUP true [get_ports PS2Clk]
44 set_property PACKAGE_PIN B17 [get_ports PS2Data]
45 set_property IOSTANDARD LVCMOS33 [get_ports PS2Data]
46 set_property PULLUP true [get_ports PS2Data]
47
48 # Don't even think to change the following values !!!!!
49 set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]
50 set_property BITSTREAM.CONFIG.CONFIGRATE 33 [current_design]
51 set_property CONFIG_MODE SPIx4 [current_design]
52 set_property CFGBVS VCC0 [current_design]
53 set_property CONFIG_VOLTAGE 3.3 [current_design]

```